

Part 2. Machine Learning

Oliver Wright

2203984

1. Introduction

Like all companies with an online presence, ScottishGlen faces a constant risk of being targeted by cyberattacks. This is even more critical now that they have received a targeted threat from a hacktivist group. Therefore, it is of utmost importance that the company's resilience to handle such attacks is thoroughly assessed. Network-based security is the priority, as it could prevent potential points of entry for damaging attacks. However, to differentiate between legitimate activity and genuine threats, the technical team would need in-depth expertise in network traffic patterns, which they may not currently possess. This is especially hard as there are many different types of attacks which can all leave a unique network footprint.

This report addresses this challenge by designing a Machine Learning (ML) classifier to categorise network packet data by attack category. The classifier will sort the packet data into 10 different categories: Normal, Fuzzers, Analysis, Backdoor, Denial of Service (DoS), Exploits, Generic, Reconnaissance, Shellcode and Worms. Following its development, this ML classifier can be integrated into ScottishGlen's network, functioning as a Network Intrusion Detection System that provides real-time detection of potential threats.

2. Background

Machine Learning for Network Security

Machine Learning algorithms can be categorised by their training methodology. Wakefield (n.d.) states "there are four types of machine learning algorithms: supervised, semi-supervised, unsupervised and reinforcement". Supervised learning is a technique that uses labelled datasets containing both input data and its target outputs to train Artificial Intelligence (AI) models. In the context of ScottishGlen, the input would be network packet data, and the output would be the correct classification of the attack state the network is in (normal, fuzzers, analysis, etc). The algorithm then learns by mapping the input to the output.

Intrusion Detection Systems

An Intrusion Detection System (IDS) is a software or hardware system that monitors data received by a network or system. The primary goal of an IDS is to use the monitored data to detect unauthorised system intrusions from malicious actors. This system often runs in real-time to catch threats early, minimising potential damage. However, an IDS can also be used post-breach for forensic analysis of an attack. According to TuxCare (2024) there are 2 main types of IDS, Network IDS (NIDS) and Host IDS (HIDS). The purpose of a NIDS is to monitor network traffic, analysing packet headers and payloads to detect potential intrusions or attacks.

Machine Learning in Intrusion Detection

Previously, Machine Learning (ML) was not used in Intrusion Detection Systems. These older IDS primarily functioned by following a set of rules (signature-based), which heavily limited their ability to reliably detect breaches. Attackers would frequently employ customised attack methods intentionally designed to evade signature-based detection methods, rendering traditional IDS ineffective. The introduction of ML in IDS was groundbreaking, as despite the unique aspects present within some of these attacks, it can still identify them as malicious. Rather than detecting threats based on predetermined rules, ML algorithms detect threats based on statistical changes and can classify traffic using characteristics learned from previous data. This allows it to continuously scale and evolve alongside new threats, automatically being able to catch them instead of temporarily leaving the network vulnerable every time a new attack is discovered.

3. Algorithm Selection and Evaluation

This section evaluates two supervised learning algorithms, Random Forest and Support Vector Machine (SVM), and assesses their suitability as a classifier for ScottishGlen's NIDS.

Algorithm 1: Random Forest

Random Forest is a supervised ensemble learning algorithm, meaning it uses labelled datasets to map input data to outputs and utilises multiple models to produce more accurate results. In other words, Random Forest combines multiple decision trees to improve its prediction accuracy, and each tree is trained on a random subset of the data. The final output is determined by taking the majority output across all trees. This process is called Bootstrap Aggregating (bagging) and can be seen in *Figure 1*. The algorithm is often used for classification or regression tasks, which makes it a practical choice for a modern NIDS.

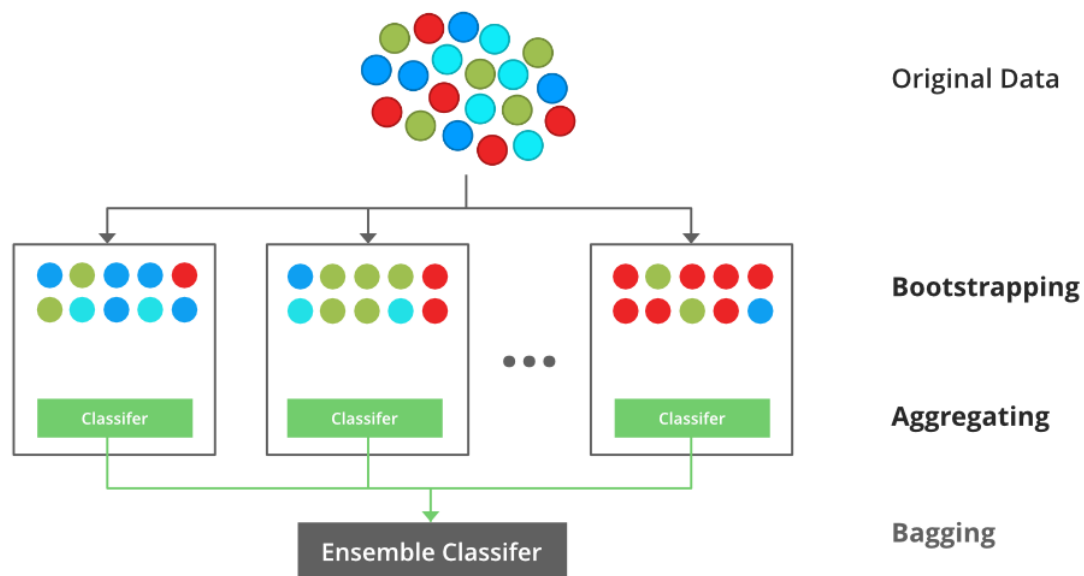


Figure 1: Bootstrap Aggregating Process (Anusha, 2022)

Most single decision tree ML algorithms are limited by their tendency to overfit training data. This limitation is addressed by Random Forest's ensemble features, which highly improve output accuracy and even allow it to maintain accurate predictions with large datasets. For the same reasons, Random Forest can also handle missing data in the datasets and continue to work accurately; this is because of a core component of how it works. Each decision tree is provided its own random dataset, so no single tree gets an entirely comprehensive view, even with a fully complete dataset. In addition, Random Forest's multiple decision tree structure improves accuracy. This is due to its method of using all results to make a final decision. With this method of reaching the final decision, a majority of trees would have to provide an inaccurate answer. While this is possible, it is much less likely to happen in an ensemble algorithm than it is for a single decision tree algorithm. Finally, the algorithm does not rely on normalisation or standardisation of the datasets, reducing the required setup preparation.

As is standard with all Machine Learning algorithms, Random Forest does still have some limitations. The algorithm's accuracy increases with the number of trees it has running. However, it also becomes more computationally expensive with a larger number of trees. This means it could be more expensive to run as accurately as possible in relation to other models. Another important component of Random Forest's accuracy is ensuring the dataset is of a large enough volume to be generously separated amongst all the decision trees. In addition to this, it has been acknowledged that Random Forest can be complex and difficult to interpret the underlying logic behind each prediction. Finally, Liedgren's work demonstrated Random Forest's ability to consistently pick up common attack types; however, the algorithm struggled with less common attacks in their methodology, such as SQL injection (2025).

Algorithm 2: Support Vector Machine

Support Vector Machine (SVM) is another supervised machine learning algorithm commonly used for classification and regression tasks. SVM separates data into classes using a boundary known as a hyperplane. It uses the data points closest to the hyperplane to determine the margin. SVM attempts to make the margin as large as possible, as this reinforces the decision and increases accuracy. While SVM is primarily used for binary classification, the one-vs-one (OvO) approach solves this, allowing it to carry out non-binary tasks. It does this by performing binary classification of each individual class against every other class, essentially running multiple binary classifiers.

While SVM specialises in binary classification, it can also be effective for multiclassification, which would be important for a NIDS. Jing & Chen (2020) researched different techniques for improving NIDSs with SVM and found that SVM with the Radial Basis Function (RBF) was essential for the task, as it massively outperformed the other two techniques. This allows SVM to transform data into a higher-dimensional space to find a stronger hyperplane. SVM excels at binary decisions, meaning that while it may not be the strongest for choice multiclassification, it has a prevalent history for being used in binary intrusion detection, producing an output of 2 choices, such as "normal" or "attack".

When given a large dataset, the SVM training process can be much slower in comparison to other machine learning algorithms. The algorithm is less equipped to handle noise present within datasets, and noisy data is likely to occur with real-world data (Al-Gethami, Al-Akhras, & Alawairdhi, 2021). This means if noisy data is present within the training process, which is common with network captures, then the algorithm may lose effectiveness. As mentioned, while primarily excelling in binary tasks, SVM can be used for multiclassification with the OvO approach. However, this can become very computationally expensive with a higher number of classes (N); in this case, there are 10 classes. The required number of running classifiers for this can be calculated as 45.

$$\frac{N * (N - 1)}{2}$$

Comparison and Selection

Resende & Drummond (2018) suggests the requirements for an IDSs machine learning capabilities "IDSs should generate low false-positive rates and deal with (i) a large amount of data for training and prediction; (ii) imbalanced datasets (malicious data are supposed to be the minority part of the dataset); (iii) a large number of features; and (iv) categorical and continuous features (e.g., protocol TCP/UDP and packet payload size)", (2018). If given the required resources, Random Forest seemingly fits these requirements. Its high accuracy reduces the likelihood of false positives; as previously stated, to produce an inaccurate result, a majority of trees would need to vote incorrectly. SVM's strengths are acknowledged, such as its ability to handle high-dimensional spaces.

However, the computational requirement of running 45 binary classifiers would be much more demanding than Random Forest's approach for handling multiclassification. Therefore, Random Forest was selected as the primary classifier for ScottishGlen's NIDS.

4. Implementation and Evaluation Metrics

Before implementation of Random Forest can occur, the dataset needs to be reviewed and modified where appropriate. The provided dataset has already been split into training data and testing data; therefore, they can be analysed, starting with the training dataset (*Figure 2*).

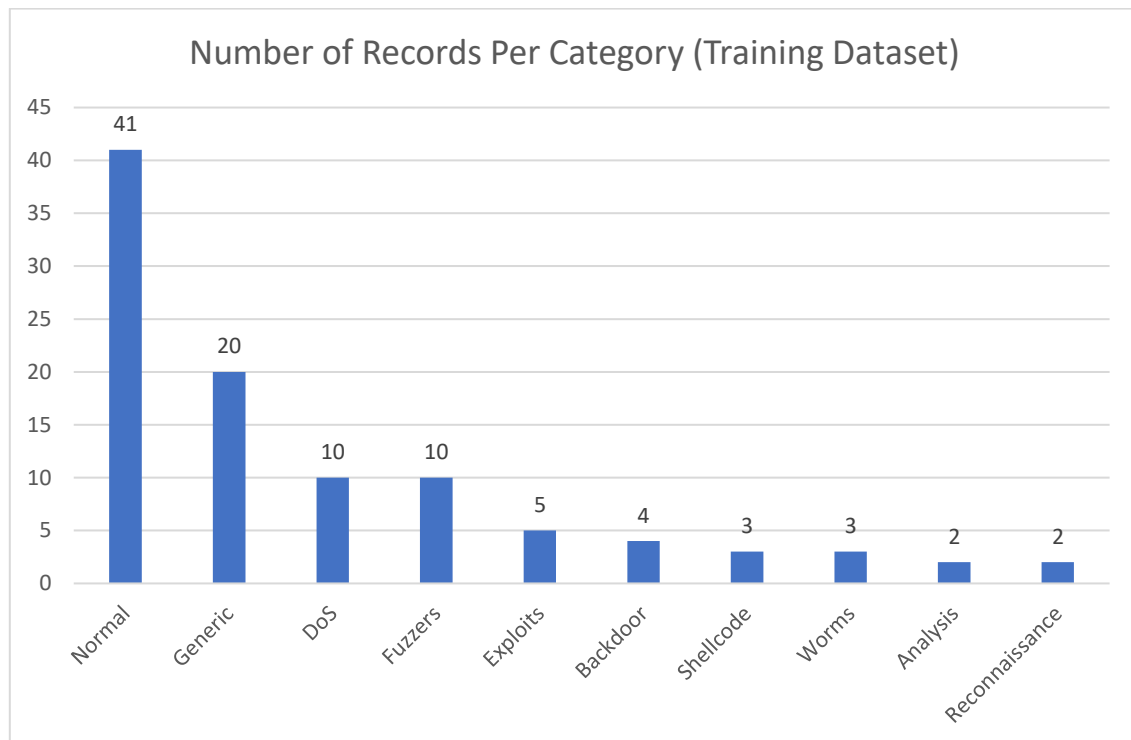


Figure 2: Chart displaying the number of records in each category (Training dataset)

The first issue to address is class imbalance within the dataset. Network capture data is usually heavily imbalanced in favour of normal traffic, outnumbering attack traffic. Due to this, less common attack classes could have so few samples that the model barely learns from them. The training dataset consists of 100 records, meaning 41% of the dataset falls into the Normal category, with Analysis and Reconnaissance containing just 2 records each, and Worms and Shellcode containing 3 each. To address this, the Synthetic Minority Oversampling Technique (SMOTE) could be implemented, increasing the volume of the less common attack classes.

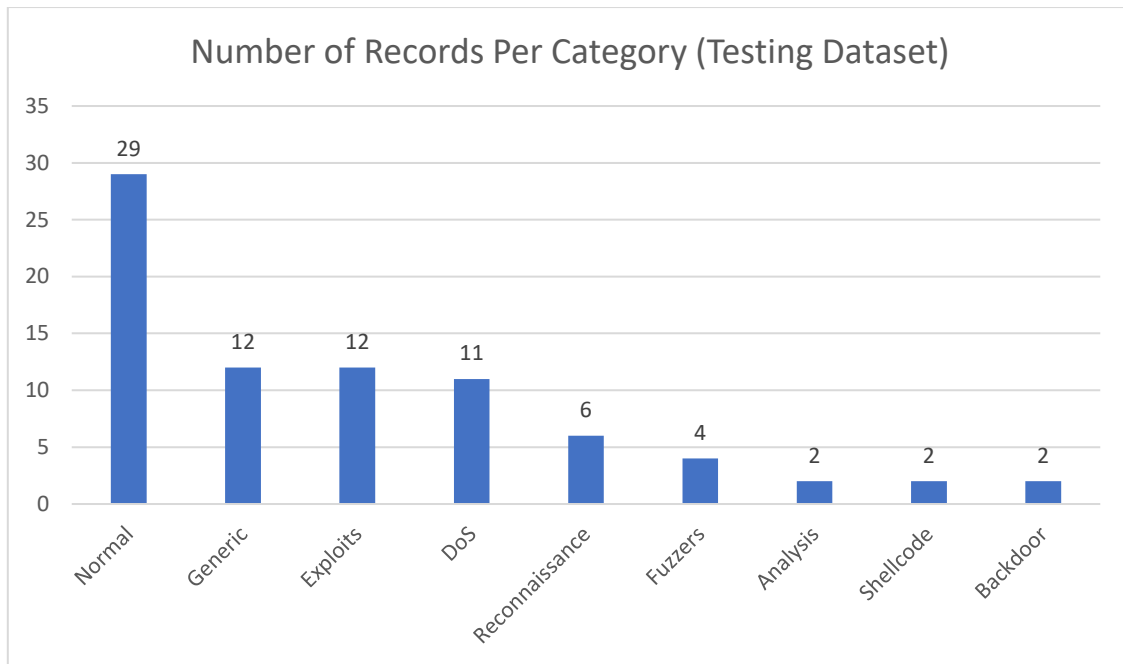


Figure 3: Chart displaying the number of records in each category (Testing dataset)

Analysis of the Testing Dataset contained a problematic lack of any records in the Worms category (Figure 3). This would greatly hinder the testing phase of the model, as it will be impossible to measure the model's capability to accurately detect Worm attacks. Similarly to the training dataset, the testing dataset would benefit from the implementation of SMOTE to add samples of Worm attacks as well as increase the number of records for the less common attack categories.

To properly implement Random Forest with accuracy and efficiency, hyperparameter tuning would need to be carried out. This is the practice of fine-tuning variables like the number of trees in the forest, improving performance but increasing computational cost. Another hyperparameter limits the number of features considered when splitting a node, helping to prevent overfitting. Additionally, a limit for the depth of each tree needs to be set. Finally, after an initial training run of the model, Random Forest's feature importance score can be used to drop redundant or low-contribution features to reduce noise present in the data.

Evaluation Metrics

Every prediction that the model makes falls into one of four outcomes: True Positive (TP), False Positive (FP), True Negative (TN) or False Negative (FN). In the context of a NIDS, this essentially identifies if the output was correct (TP/TN), a false alarm (FP), or a missed attack (FN). This is an important aspect of building any machine learning technology, as it allows its performance to be evaluated.

Accuracy is a common metric and is calculated by finding the percentage of correct decisions. However, as a standalone metric, it can be misleading as the NIDS, for example, would score a 36% accuracy by predicting normal for every result.

$$Accuracy = \frac{TP + TN}{TP + TN + FP + FN}$$

Precision is another metric which measures the model's positive prediction correctness. This is useful to monitor as false positives in a NIDS will be treated as real attacks and can be expensive both in terms of finances and time to handle and identify.

$$Precision = \frac{TP}{TP + FP}$$

Recall measures how many positive cases were identified. In other words, it calculates the percentage of attacks caught by the model; a lower recall means that more attacks went unnoticed. This is even more critical than the model producing false positives as it could leave the company vulnerable to attacks without even noticing.

$$Recall = \frac{TP}{TP + FN}$$

F1-Score is the harmonic mean of precision and recall. It combines the two metrics into one, providing a more general analysis of the model. However, for the 10-class problem at hand, the Macro-Averaged F1-Score would provide the strongest standalone value for measuring performance. This takes the F1-Score of each class and averages them, ensuring even the classes with as few as 2 records share the same importance as the 41-record Normal class.

$$F1\ Score = 2 \left(\frac{Precision * Recall}{Precision + Recall} \right)$$

A confusion matrix is another strong metric for measuring performance of a classifier. It is an N*N grid, where N is the number of classes, which categorises the model's results into a comprehensive structure. It displays the number of outputs for each of the four potential success outcomes (TP, FP, TN, FN) for each category. This would show exactly where the model is struggling while structuring the information to calculate the previous metrics.

Together, these metrics provide the ability to thoroughly analyse classifier performance across all 10 attack categories, ensuring both common and rare attack types are evaluated equally.

References

- Al-Gethami, K., Al-Akhras, M., & Alawairdhi, M. (2021, January 15). *Empirical Evaluation of Noise Influence on Supervised Machine Learning Algorithms Using Intrusion Detection Datasets*. Retrieved from Wiley: <https://onlinelibrary.wiley.com/doi/10.1155/2021/8836057>
- Anusha, H. (2022, January 28). *Bootstrapped Aggregation(Bagging)*. Retrieved from Medium: <https://medium.com/@hemaanushatangellamudi/bootstrapped-aggregation-bagging-481f4812e3ea>
- Jing, D., & Chen, H.-B. (2020, February 6). *SVM-Based Network Intrusion Detection for the UNSW-NB15 Dataset*. Retrieved from ieeexplore.ieee.org: <https://ieeexplore.ieee.org/document/8983598>
- Liedgren, J. (2025). *Comparison of Machine-Learning Algorithms for Intrusion Detection Systems*. Stockholm: Stockholm University. Retrieved from <https://urn.kb.se/resolve?urn=urn:nbn:se:su:diva-244450>
- Resende, P. A., & Drummond, A. C. (2018, May 23). *A Survey of Random Forest Based Methods for Intrusion Detection Systems*. Retrieved from ACM: <https://dl.acm.org/doi/10.1145/3178582>
- TuxCare. (2024, August 9). *7 Types of Intrusion Detection Systems that Identify Network Anomalies*. Retrieved from TuxCare: <https://tuxcare.com/blog/types-of-ids/>
- Wakefield, K. (n.d.). *A guide to the types of machine learning algorithms and their applications*. Retrieved from SAS: https://www.sas.com/en_gb/insights/articles/analytics/machine-learning-algorithms.html